SECURE: Secure and Efficient Protocol Using Randomness and Edge-Computing for Drone-Assisted Internet of Vehicles

Himani Sikarwar, Harsha Vasudev, *Student Member, IEEE*, Debasis Das, *Senior Member, IEEE*, Mauro Conti, *Fellow, IEEE*, and Koustav Kumar Mondal

***Abstract*—The Internet of Vehicles (IoV) faces significant challenges related to secure authentication, efficient communication, and privacy preservation due to the high mobility of vehicles, the need for real-time data processing, varying quality of communication links, and the diverse range of devices and protocols requiring interoperability. These challenges are further complicated by the large-scale, dynamic, and heterogeneous nature of IoV systems. Traditional approaches using Road Side Connecting Nodes (RSCNs) face challenges like limited range, high costs, and single points of failure. Drone-assisted IoV (DIoV) networks address these issues by using Unmanned Aerial Vehicles (UAVs) as mobile edge nodes, enhancing connectivity, extending coverage, and improving adaptability and resilience. To address these challenges, we propose *SECURE*, a drone-assisted, Physically Unclonable Function (PUF)-based authentication and privacy-preserving protocol integrated with edge computing. This architecture replaces RSCNs with edge nodes and incorporates UAVs as mobile edge nodes, providing extended coverage, reduced latency, and enhanced adaptability. The PUFs in *SECURE* generate unique hardware-based cryptographic keys, adding an additional layer of security, while edge computing offloads computational tasks, improves network efficiency, and further reduces latency. The formal security analysis, conducted using the Random Oracle Model (ROM), proves the robustness of the session key against active and passive adversaries. Furthermore, informal security analysis demonstrates that *SECURE* effectively resists various security attacks, while achieving confidentiality, integrity, and authenticity in DIoV. In *SECURE*, we have considered two types of devices for experiments: NVIDIA Jetson Xavier NX and Raspberry Pi 4. The performance analysis, considering the results from Jetson Xavier NX, demonstrates that *SECURE* achieves maximum upto approximately 82.1% less communication cost and 78% faster computation time compared to the state-of-the-art schemes.**

***Index Terms*—Internet of Vehicles, security, edge-computing, physical unclonable function, drone-assisted IoV.**

Received 26 April 2024; revised 19 August 2024; accepted 4 September 2024. Date of publication 17 September 2024; date of current version 20 December 2024. This work is partially supported by MeitY (Project No.-AAA.2/12/2021-CSRD-MeitY), TIH IoT and IoE Project (Project No.: TDP06-A-26), and CY4GATE. The associate editor coordinating the review of this article and approving it for publication was R. Doriguzzi-Corin. *(Corresponding author: Debasis Das.)*

Himani Sikarwar and Debasis Das are with the Department of Computer Science and Engineering, Indian Institute of Technology Jodhpur, Jodhpur 342001, India (e-mail: himani.1@iitj.ac.in; debasis@iitj.ac.in).

Harsha Vasudev and Mauro Conti are with the HIT Research Centre, Department of Mathematics, University of Padua, 35122 Padua, Italy (e-mail: harsha.vasudevanpillai@unipd.it; mauro.conti@unipd.it).

Koustav Kumar Mondal is with the Inter-Disciplinary Research Division - IoT and Application, Indian Institute of Technology Jodhpur, Jodhpur 342001, India (e-mail: mondal.4@iitj.ac.in).

Digital Object Identifier 10.1109/TNSM.2024.3462746

I. Introduction

IN RECENT years, the rapid expansion of the Internet of Vehicles (IoV) has revolutionized the transportation industry, promising a future of enhanced connectivity, autonomous driving, and efficient traffic management [4]. However, as IoV networks grow in complexity and scale, they also face significant security challenges. According to a report by the National Highway Traffic Safety Administration (NHTSA) [1], over 31,000 fatalities were recorded in traffic crashes in the first nine months of 2022, underscoring the critical need for secure and reliable communication within vehicular networks. The IoV environment, characterized by its high mobility, real-time data processing requirements, and the diverse range of devices it connects, presents a unique set of vulnerabilities [3]. These include risks associated with secure authentication, efficient communication, and privacy preservation, which are exacerbated by the dynamic and heterogeneous nature of IoV systems. As the need for robust security in IoV systems becomes increasingly apparent, researchers have begun exploring innovative solutions to mitigate these challenges. One promising approach is the integration of Unmanned Aerial Vehicles (UAVs) or drones into IoV networks, giving rise to the concept of the Drone-assisted Internet of Vehicles (DIoV) [8]. The Internet of Drones (IoD) [6], an extension of IoV, leverages the unique capabilities of drones to enhance network coverage, improve data collection, and offer real-time surveillance. Drones, equipped with advanced sensors, communication modules, and computing power, can serve as mobile edge nodes within IoV networks, providing critical support in scenarios where traditional Road Side Connecting Nodes (RSCNs) may fall short [2].

The integration of drones into IoV networks to create DIoV offers several distinct advantages over traditional IoV. For instance, drones can provide real-time aerial surveillance, monitor environmental conditions, and act as aerial base stations, ensuring connectivity in areas with weak or no signal [5]. Moreover, drones can be deployed rapidly in emergency situations, providing immediate visuals and communication support to first responders. This flexibility and adaptability make DIoV a superior solution for enhancing the resilience and efficiency of IoV networks [7]. However, despite the potential of DIoV, existing security methods [16], [17], [18] face significant challenges in this dynamic environment [10].

1932-4537 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

Traditional protocols, designed for IoV, are inadequate for DIoV due to their reliance on RSCNs, which are limited by communication range, high deployment costs, and vulnerability to single points of failure. These protocols also struggle with scalability issues, as the high mobility of drones introduces latency and computational inefficiencies [8]. Moreover, current security measures are often too resource-intensive for lightweight devices like drones, failing to provide robust protection against impersonation, replay, and brute-force attacks. These limitations highlight the need for a new security framework tailored to the unique demands of DIoV, ensuring both efficiency and resilience against emerging threats [9], [12]. To enhance security, one of the emerging trends gaining attention from both industry and academia is the implementation of Physical Unclonable Function (PUF) [11]. One of the key advantages of PUF is its ability to provide device-specific randomness and robust protection against tampering, all while eliminating the need for key storage. Although existing schemes [21], [22], [23], [24] offer security against various drone attacks, issues related to network delay and performance remain as open challenges.

To address these challenges, we propose *SECURE*, a drone-assisted authentication and privacy-preserving protocol that integrates PUF with edge computing. In *SECURE*, drones are deployed within specific zones to assist RSCNs during network bottlenecks, enhancing overall performance by extending network coverage, reducing latency, and improving adaptability. PUFs provide lightweight, hardware-based cryptographic key management, enhancing security without relying on traditional key storage. Additionally, edge computing offloads computational tasks to local nodes, improving efficiency and reducing communication costs. The key contributions of this research are listed below:

- We integrate PUF with edge computing in DIoVs to enable secure, hardware-based cryptographic key generation. This approach leverages the non-replicable nature of PUFs to dynamically generate session keys, significantly mitigating risks such as impersonation, replay, and brute-force attacks without reliance on static key storage.
- The *SECURE* protocol introduces a robust privacy-preserving authentication mechanism that leverages PUF responses for dynamic identity and session key management. This design ensures mutual authentication while providing strong resistance against eavesdropping and man-in-the-middle attacks, thereby enhancing forward and backward secrecy.
- We propose a scalable security architecture that seamlessly integrates drones and smart vehicles into DIoVs. The dynamic PUF-based identity verification and session key management adapt to network topology changes, ensuring efficient and secure communication in large-scale, real-time deployments with low computational overhead.
- A formal security analysis using the Random Oracle Model (ROM) validates the protocol's resilience to advanced cryptographic attacks, confirming the robustness of *SECURE* in maintaining the confidentiality, integrity, and authenticity of DIoV communications.
- *SECURE* is implemented on NVIDIA Jetson Xavier NX and Raspberry Pi 4 platforms, demonstrating its practical applicability. Experimental evaluations show marked improvements in security and performance, including reduced latency and communication overhead, making *SECURE* an effective solution for DIoV security.

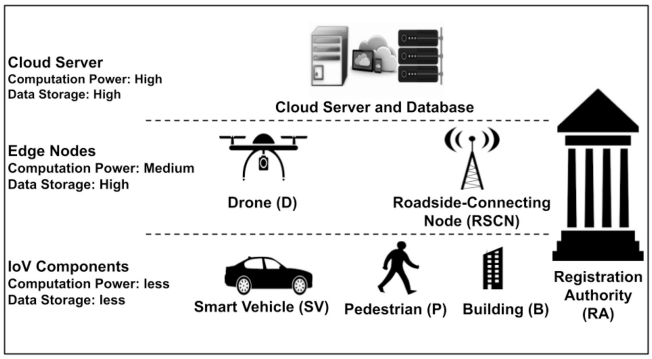


Fig. 1. Layers and DIoVs Components of *SECURE* Protocol.

The Fig. 1 illustrates the layered architecture of the *SECURE* protocol within a DIoV network, focusing on the efficient distribution of computational tasks and enhanced security. At the base layer, IoV components like Smart Vehicles (SV), pedestrians, and smart buildings have limited computational power and storage, relying on the intermediate layer of edge nodes–drones and RSCNs. These edge nodes, with medium computational power and substantial storage, handle data processing closer to the source. Drones assist RSCNs during bottlenecks, extending coverage and ensuring low latency. The top layer, featuring a Cloud Server with high computational power and storage, manages large-scale data processing and supports the Registration Authority (RA) in identity and authentication management. This layered approach in the *SECURE* protocol ensures efficient real-time operations by edge nodes and centralized control by the Cloud Server, maintaining a secure and scalable DIoV network.

II. Related Work

We extensively studied authentication protocols related to DIoV and identified several gaps in existing works, particularly regarding security, scalability, and computational efficiency. The protocol by [13] for IoT environments is lightweight, using bit-wise XOR and hash functions, but it is vulnerable to impersonation, stolen smart cards, and man-in-the-middle attacks due to the simplicity of XOR operations, which do not provide robust security. Moreover, it lacks critical security features like sensing node anonymity, user untraceability, and session key security, essential in DIoV environments. Farash et al. [14] improved upon [13] by using session keys for secure communication, but their scheme still suffers from offline password guessing attacks and impersonation due to reliance on static passwords. Reference [15] introduced a signature-based user authentication protocol using fuzzy

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF AUTHENTICATION PROTOCOLS IN DIOVS AND RELATED FIELDS

Protocol	Security Features	Limitations	Scalability	Efficiency
Turkanović et al. [2014] [13]	Lightweight, XOR and hash functions	Vulnerable to impersonation, lacks anonymity and session key	Low	High (lightweight)
Farash et al. [2016] [14]	Session keys, improved security	Still vulnerable to offline attacks, impersonation	Moderate	Moderate
Challa et al. [2017] [15]	Signature-based, ECC, fuzzy extractors	High computation and communication costs	Moderate	Low
Won et al. [2017] [19]	Certificateless, multi-recipient encryption, data aggregation	Complex, resource-demanding	High (for small deployments)	Moderate
Wazid et al. [2018] [20]	Lightweight, direct drone data access	Not resistant to stolen verifier and traceability attacks	Low	High (lightweight)
Khan et al. [2021] [21]	Privacy-preserving, HECC	Not scalable	Low	Moderate
Hussain et al. [2022] [22]	Lightweight, improved security	High communication and computation costs	Moderate	Low (compared to SECURE)
Wu et al. [2022] [23]	Anonymity, identity tracing	High communication and computation costs	Low (for large deployments)	Low
Chatterjee et al. [2018] [25]	ECC, universal composability	Potential vulnerability due to reliance on cryptographic primitives	Moderate	Moderate
Wang et al. [2024] [26]	Three-factor authentication, PUFs	High computational overhead	Moderate	Low (in resource-constrained environments)
Karmakar et al. [2024] [27]	Mutual authentication, PUFs	Requires synchronized clocks, potential synchronization issues	Low (in heterogeneous environments)	Moderate
Tian et al. [2024] [28]	Perfect forward secrecy, device anonymity	Complex, scalability and efficiency challenges	Low (in large-scale deployments)	Low
SECURE (Proposed)	PUF-based, edge computing, scalable, dynamic key management	Addresses limitations in existing protocols, low overhead	High (scalable for large networks)	High (efficient and lightweight)

extractors for biometric verification and ECC for signature mechanisms. While enhancing security, the high computational and communication costs of ECC and fuzzy extractors limit its applicability in resource-constrained DIoVs, where such delays are undesirable. Won et al. [19] proposed protocols for various communication scenarios in DIoVs that reduce computational overhead, but the complexity and resource demands of certificateless schemes can pose challenges in large-scale deployments due to the need to manage multiple certificates and keys. The protocol by [20] introduces a lightweight user authentication method for IoD environments but is vulnerable to stolen verifier and traceability attacks, raising concerns about privacy and data integrity. Khan et al. [21] designed a privacy-preserving authentication scheme for UAV-enabled ITS using digital signatures, hash functions, and HECC techniques. While secure, it is not scalable, making it unsuitable for large DIoV networks due to significant computational costs. Hussain et al. [22] proposed an authentication scheme for smart city surveillance that addresses some flaws of [20], but it incurs higher communication and computation costs, limiting its efficiency in large networks. Wu et al. [23] developed an edge computing-enabled authentication scheme with anonymity and identity tracing, but its high communication and computation costs are drawbacks for real-time DIoV applications. Chaudhry et al. [24] proposed a lightweight authentication scheme for 6G-IoT maritime transport systems, but its communication and computation costs are higher than those of SECURE due to reliance on multiple cryptographic operations. Chatterjee et al. [25] introduced a protocol leveraging ECC and universal composability, but its reliance on complex cryptographic primitives could present vulnerabilities in large-scale IoT deployments. Wang et al. [26] proposed a three-factor authentication protocol using biometrics, passwords, and PUFs, which enhances security but faces challenges related to high computational overhead, limiting practicality in DIoVs. Karmakar et al. [27] explored a mutual authentication protocol that resists various attacks but requires synchronized clocks across the network, which can lead to issues in dynamic IoT environments. Tian et al. [28] presented a protocol focusing on perfect forward secrecy and device anonymity using PUFs, but its complexity and reliance on multiple cryptographic operations pose scalability and efficiency challenges in large-scale IoT deployments, making it less suitable for large, dynamic networks like DIoVs.

Further, to provide more clarity, the Table I visualizes the strengths and limitations of the above discussed protocols, particularly in terms of their security features, scalability, computational efficiency, and applicability to DIoV environments.

III. MATHEMATICAL PRELIMINARIES

In this section, the basic formulations for the PUF function, PUF stability and the Error Correction Codes (ECC) have been demonstrated.

A. Physical Unclonable Functions

A PUF function is a mathematical function denoted as f, which maps a set of challenges C to a set of responses R in a physically unique and unpredictable manner. This relationship can be expressed as: $f : C \rightarrow R$, where $C = \{c_1, c_2, \ldots, c_n\}$ represents the set of challenges and $R = \{r_1, r_2, \ldots, r_n\}$ represents the corresponding set of responses.

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

1) *Key Properties of PUFs:* The key properties of PUFs are described below:
   - *Uniqueness:* Each challenge c_i generates a unique response r_i, such that $f(c_i) = r_i$.
   - *Unpredictability:* It is computationally infeasible to predict r_i for a given c_i without physically interacting with the PUF.
   - *Reliability:* The PUF consistently produces the same response for the same challenge under varying environmental conditions.
   - *Non-Clonability:* Duplicating the PUF or deriving an equivalent function is computationally infeasible due to the inherent manufacturing variations in the physical structure of the device.
   - *Noise Tolerance:* PUFs are designed to be robust against environmental variations such as temperature, aging, and power supply fluctuations.
2) *Mathematical Representation of PUFs:* For a given challenge c_i, the response r_i can be mathematically represented as: $r_i = \mathrm{PUF}(c_i)$, where $\mathrm{PUF}(\cdot)$ is the unique function of each physical instance of the PUF. The response r_i is utilized in various cryptographic operations within the proposed protocol to ensure secure authentication and key management.

B. Error Correction Codes (ECCs) and PUF Stability

PUFs are inherently sensitive to environmental conditions such as temperature, voltage variations, and aging. These factors can cause bit errors in the responses generated by PUFs, potentially compromising the security and reliability of cryptographic protocols that depend on them. To mitigate these issues, the *SECURE* protocol employs robust Error Correction Codes (ECCs), specifically focusing on Bose–Chaudhuri–Hocquenghem (BCH) codes, which are a class of cyclic ECCs known for their strong error correction capabilities.

1) *BCH Code Implementation:* BCH codes are well-suited for correcting multiple random errors in binary sequences. In the context of *SECURE*, the PUF generates a binary response *R* in response to a challenge *C*. Given the possibility of bit errors, we encode this response using a BCH code before storing or transmitting it. For example, consider a 64-bit PUF response $R = r_1, r_2, \ldots, r_{64}$. This response is encoded using a BCH(127,64) code, which maps the 64-bit response to a 127-bit codeword. This codeword can correct up to 10 random bit errors, providing a significant level of robustness against environmental variations. The encoded response $R_{encoded}$ is then used in the subsequent cryptographic operations. The encoding process can be mathematically represented as: $R_{encoded} = \mathrm{BCH_encode}(R)$, where $R_{encoded}$ is the 127-bit codeword generated by the BCH encoding process.
2) *Syndrome Decoding and Error Correction:* When the encoded PUF response $R_{encoded}$ is retrieved or received, it may contain bit errors due to environmental influences. The *SECURE* protocol applies syndrome decoding to detect and correct these errors. The syndrome is calculated from the received codeword, and if the syndrome is non-zero, it indicates the presence of errors. The decoding process involves the following steps:
   a) *Syndrome Calculation:* The syndrome *S* is computed using the received codeword. The syndrome is a vector that represents the difference between the received codeword and a valid codeword in the BCH code space.
   b) *Error Detection:* If the syndrome *S* is non-zero, it indicates that errors are present in the received codeword.
   c) *Error Correction:* The BCH decoder uses the syndrome to identify the location and magnitude of the errors in the codeword. It then corrects these errors to recover the original PUF response *R*.

   The corrected response is represented as:

$$R_{corrected} = \mathrm{BCH_decode}(R_{received})$$

   where $R_{received}$ is the potentially corrupted codeword and $R_{corrected}$ is the error-free, recovered PUF response.

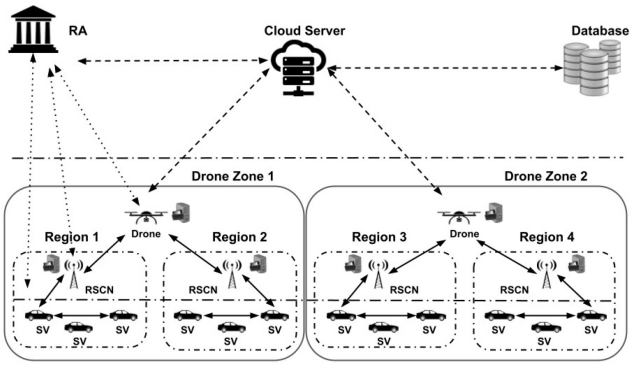


Fig. 2. Network Model of *SECURE*.

IV. Network and Adversary Model

This section outlines the network architecture of the *SECURE* protocol, detailing the roles of its components, and defines the adversary model, capturing potential threats and attack strategies. These foundations are crucial for understanding the security measures implemented in the protocol.

A. Network Model

The network model of *SECURE*, as depicted in Fig. 2, comprises several key components: SV as Vehicular Nodes (VN), drones and RSCNs as edge nodes, RA, and a Cloud Server (CS). In this model, RSCNs are strategically deployed across different regions, with each drone overseeing a specific zone. The RA, as a fully trusted entity, is responsible for the secure registration and management of SVs, drones, and RSCNs, including maintaining a database of identities, essential keys, and other critical information. In cases of security breaches or vehicle theft, the RA has the authority to trace and revoke certificates.

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

Drones and RSCNs function as intelligent edge nodes within the *SECURE* framework. Drones are equipped with unclonable PUF devices, advanced navigation and communication systems, and are tasked with bridging communication gaps between RSCNs, transferring data to the Cloud Server, and performing mutual authentication. RSCNs, on the other hand, are deployed along road networks and act as fixed edge nodes equipped with PUF chips, wireless communication modules, and sufficient computational resources. They facilitate communication between vehicles, drones, and the Cloud Server, effectively partitioning the road network into distinct zones to manage communication requests efficiently.

SVs, as dynamic vehicular nodes, are equipped with On-Board Units (OBU), PUF devices, and wireless communication modules, enabling secure communication within their designated zones via the IEEE 802.11p protocol. Each SV undergoes a pre-registration process, during which it receives a smart card with a unique ID and password, ensuring secure interactions with other entities in the DIoV environment, including RSCNs, drones, pedestrians, and buildings. The Cloud Server plays a central role, managing PUF keys, digital certificates, and authentication requests, and ensuring secure communication across the network by interacting with drones, RSCNs, SVs, and the RA. This comprehensive and layered approach ensures efficient and secure management of the DIoV network within the *SECURE* protocol.

B. Adversary Model and Design Goals

In this work, we adopt a comprehensive adversary model that encompasses three well-established frameworks: the Canetti-Krawczyk (CK) model, the Dolev-Yao (DY) model, and the extended Canetti-Krawczyk (eCK) model [29]. Each of these models provides a distinct perspective on the potential capabilities of adversaries and the types of threats that a cryptographic protocol must be robust against. CK model is a widely recognized framework for analyzing key exchange protocols. It is particularly focused on ensuring security properties such as secrecy and authenticity, even in the presence of active adversaries who can manipulate the protocol's execution. Unlike the CK model, the DY model operates under the assumption that the adversary has complete control over the communication network. This includes the ability to intercept, delete, alter, and fabricate messages. The DY model abstracts cryptographic operations as closed boxes, making it ideal for formal verification and symbolic analysis of protocols. Further, eCK model is designed to handle adversaries who can compromise long-term keys and still ensure the security of session keys established before the compromise. It introduces the concept of forward secrecy and is particularly concerned with ensuring security even if some of the protocol participants' long-term secrets are exposed. Table II describes the possible tasks that can be performed by an adversary $\mathcal{A}$ under the CK, DY and eCK adversary model. Further, we have considered the following design goals of *SECURE* under the respective adversary models:

- *Secrecy:* Achieve confidentiality and protect sensitive information from unauthorized disclosure. It is denoted as $\Pr(Secret = s, |, Ciphertext = c) \approx \Pr(Secret = s)$, where *Secret* represents the sensitive information, *Ciphertext* represents the encrypted data, and *s* represents a possible value of the secret.
- *Authentication:* This property ensure the authenticity and integrity of participants' identities or messages. It is denoted as $\Pr(Authenticity = true, |, M = m) \approx 1$, where *Authenticity* represents the verification of a participant's identity, *M* represents the transmitted message and *m* represents a possible message value.
- *Integrity:* This property guarantees that data remains unaltered and tamper-proof during transmission or storage. It is denoted as $\Pr(Integrity = true, |, ReceivedData = d) \approx 1$, where *Integrity* represents the verification of data integrity, *ReceivedData* represents the received data, and *d* represents a possible data value.
- *Non-repudiation:* Prevent participants from denying their involvement or actions in a communication or transaction. It is denoted as $\Pr(NonRepudiation = true, |, Sender = s, M = m) \approx 1$, where *NonRepudiation* represents the ability to prove the sender's involvement, *Sender* represents the participant who sent the message, and *M* represents the transmitted message.
- *Availability:* Ensure continuous and timely access to resources or services and denoted as $\Pr(Availability = true) \approx 1$, where *Availability* represents the ability to access resources or services without interruption.
- *Freshness/Replay Protection:* Prevent replay attacks by ensuring the freshness of messages or data. It is denoted as $\Pr(Freshness = true, |, Message = m) \approx 1$, where *Freshness* represents the verification of message freshness, and *m* represents a possible message value.
- *Forward Secrecy:* Protect past session keys if the long-term key is compromised. It is denoted as $\Pr(ForwardSecrecy = true) \approx 1$, where *ForwardSecrecy* represents the ability to maintain the confidentiality of past session keys.
- *Backward Secrecy:* This property protects future session keys if the long-term key is compromised. It is denoted as $\Pr(BackwardSecrecy = true) \approx 1$ where *BackwardSecrecy* represents the ability to maintain the confidentiality of future session keys.
- *Resistance to Brute-Force Attacks:* Withstand exhaustive search attacks against cryptographic keys and denoted as $\Pr(BruteForceResistance = true) \approx 1$.
- *Privacy Preservation:* We have to preserve the privacy of individuals and sensitive information. It is denoted as $\Pr(PrivacyPreservation = true) \approx 1$.
- *Minimization of Information Leakage:* Limit the amount of information revealed to unauthorized parties. It is denoted as $\Pr(InformationLeakage = low) \approx 1$.

V. *SECURE*: Our Proposed Protocol

This section outlines the *SECURE* protocol, a robust framework designed to enhance the security and efficiency of communication within a DIoV network. This section details the sequential processes of registration and authentication for

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

TABLE II
ADVERSARY CAPABILITIES IN SECURE PROTOCOL

Capability	Description
Message Interception (DY Model)	$\mathcal{A}$ can intercept, eavesdrop, and modify any message exchanged in the network. This is represented as: $\mathcal{A}.\texttt{Intercept}(M) \Rightarrow M' = \mathcal{A}.\texttt{Modify}(M)$, where M is the original message and M' is the modified message.
Message Modification (CK and eCK Models)	$\mathcal{A}$ can modify messages or inject new messages into the communication. This is represented as: $\mathcal{A}.\texttt{Modify}(M_1, M_2) \Rightarrow M_2 = \mathcal{A}.\texttt{Inject}(M_1, \text{New_Data})$, where M_1 is the intercepted message, and M_2 is the modified message with injected data.
Active Attacks (eCK Model)	$\mathcal{A}$ can impersonate participants and perform man-in-the-middle attacks. This is represented as: $\mathcal{A}.\texttt{MitM}(V_i, V_j) \Rightarrow \mathcal{A}(V_j) = \mathcal{A}.\texttt{Send}(V_i, \text{Forged_Data})$, where V_i and V_j are legitimate participants.
Replay Attacks (DY and CK Models)	$\mathcal{A}$ can replay previously intercepted messages to disrupt the protocol execution. This is represented as: $\mathcal{A}.\texttt{Replay}(M, T) \Rightarrow M' = \mathcal{A}(M, \texttt{Timestamp}(T))$, where T is the timestamp associated with the original message M.
Computational Power	$\mathcal{A}$ is assumed to have polynomial-time computational power, enabling brute-force and other cryptographic attacks.
Cryptographic Assumptions	$\mathcal{A}$ cannot break the underlying cryptographic primitives used in the protocol, such as encryption, digital signatures, or hash functions.

TABLE III
NOTATIONS AND THEIR EXPLANATIONS

Notation	Explanation
SV	Smart Vehicle
RA	Registration Authority
RSCN	Road Side Connecting Nodes
$\mathcal{D}$	Drone
ID_d	Drone's identity
Req_d	Registration request from drone
AID_d	Unique Anonymous Identity for drone
$\mathcal{C}_d$	Random Challenge generated by RA for drone
$\mathcal{C}_d^{sync}$	Set of re-synchronization challenges generated by RA for drones
$\mathcal{P}_d$	PUF function of drone
Req_r	Registration request from RSCN
$\mathcal{C}_r$	Random Challenge generated by RA for RSCN
$\mathcal{C}_r^{sync}$	Set of re-synchronization challenges generated by RA for RSCN
UID_r	Unique identity generated by RA for RSCN
ID_{sv}	Identity of SV
Req_{sv}	Registration request from SV
$\mathcal{C}_{sv}$	Random Challenge generated by RA for SV
$\mathcal{C}_{sv}^{sync}$	Set of re-synchronization challenges generated by RA for SV
AID_{cv}	Unique Anonymous Identity for SV
$\mathcal{P}_{sv}$	PUF Function of SV

various network components, including drones, RSCNs, and SVs. The required notations and their meaning are summarized in Table III.

A. Registration Process

This subsection details the initial steps required to securely onboard various components of the DIoV network, including drones, RSCNs, and SV. This process is crucial for establishing unique identities and cryptographic credentials, ensuring that each entity is authenticated and ready for secure communication within the network.

1) *Drone Registration:*
   - The drone, $\mathcal{D}$ sends its identity, ID_d and registration request, Req_d via a secure medium to the RA.
   - As depicted in the Algorithm 1, RA generates a random challenge, $\mathcal{C}_d$ and a set of re-synchronization challenges, $\mathcal{C}_d^{sync} = \{c_1, c_2, \ldots, c_n\}$ to avoid Denial-of-Services (DoS) attack and synchronization issues. The RA sends $\mathcal{C}_d$ and $\mathcal{C}_d^{sync}$ to the corresponding drone.
   - After receiving the challenges, $\mathcal{D}$ uses its PUF function $(\mathcal{P}_d)$ and generates the required response as: $\mathcal{R}_d = \mathcal{P}_d(\mathcal{C}_d)$ and $\mathcal{R}_d^{sync} = \mathcal{P}_d(\mathcal{C}_d^{sync})$. Now, $\mathcal{D}$ sends $\{\mathcal{R}_d, \mathcal{R}_d^{sync}\}$ to RA via a secure medium.
   - Upon receiving the responses, RA generates a unique anonymous identity for $\mathcal{D}$ as AID_d. The RA sends AID_d to the corresponding $\mathcal{D}$ and saves $\{(AID_d, ID_d, (\mathcal{C}_d, \mathcal{R}_d), \text{ and } (\mathcal{C}_d^{sync}, \mathcal{R}_d^{sync})\}$. Simultaneously, $\mathcal{D}$ saves $\{AID_d\}$.

2) *RSCN Registration:*
   - Initially, RSCN sends the registration request (Req_r) to RA through a secure medium.
   - Upon receiving the registration request, RA randomly generates the challenge $\mathcal{C}_r$ and a set of re-synchronization challenges $\mathcal{C}_r^{sync}$. The RA sends these challenges $\{\mathcal{C}_r, \mathcal{C}_r^{sync}\}$ to the RSCN via a secure medium.
   - As described in the Algorithm 1, the RSCN computes a response and a set of re-synchronization responses corresponding to the received challenges using the PUF function as, $\mathcal{R}_r = \mathcal{P}_r(\mathcal{R}_r)$ and $\mathcal{R}_r^{sync} = \mathcal{P}_r(\mathcal{R}_r^{sync})$. The RSCN sends $\{\mathcal{R}_r, \mathcal{R}_r^{sync}\}$ to the RA via a secure medium.
   - Now, RA randomly generates a unique identity for RSCN as UID_r and sends it to the RSCN. Also, it stores $\{UID_r, (\mathcal{C}_r, \mathcal{R}_r)$, and $(\mathcal{C}_r^{sync}, \mathcal{R}_r^{sync})\}$ in its database for further references.

3) *Smart Vehicle Registration:*
   - Initially, *SV* sends ID_{sv} and Req_{sv} via a secure medium to the RA.
   - As shown in the Algorithm 1, RA generates the a random challenge, $\mathcal{C}_{sv}$ and a set of re-synchronization challenges, $\mathcal{C}_{sv}^{sync} = \{c_1, c_2, \ldots, c_n\}$ to avoid DoS attack and synchronization issues. The RA sends $\mathcal{C}_{sv}$ and $\mathcal{C}_{sv}^{sync}$ to the corresponding *SV*.
   - After receiving the challenges, *SV* uses its PUF function, $\mathcal{P}_{sv}$ and generates the required response as: $\mathcal{R}_{sv} = \mathcal{P}_{sv}(\mathcal{C}_{sv})$ and $\mathcal{R}_{sv}^{sync} = \mathcal{P}_{sv}(\mathcal{C}_{sv}^{sync})$. Now, *SV* sends $\{\mathcal{R}_{sv}, \mathcal{R}_{sv}^{sync}\}$ to RA via a secure medium.
   - Upon receiving the responses, RA generates a unique anonymous identity for *SV* as AID_{sv}. RA

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Combined Registration Process for Drone, RSCN, and Smart Vehicle

Input: ID_d, Req_d, $Reqr$, ID_{sv}, Req_{sv}
Output: AID_d, UID_r, AID_{sv}
Drone Registration:
$\mathcal{C}_d \leftarrow$ generate random challenge;
$\mathcal{C}_d^{sync} \leftarrow$ generate set of re-synchronization challenges;
RA sends $\mathcal{C}_d$ and $\mathcal{C}_d^{sync}$ to $\mathcal{D}$;
$\mathcal{R}_d \leftarrow \mathcal{P}_d(\mathcal{C}_d)$;
$\mathcal{R}_d^{sync} \leftarrow \mathcal{P}_d(\mathcal{C}_d^{sync})$;
$\mathcal{D}$ sends $\{\mathcal{R}_d, \mathcal{R}_d^{sync}\}$ to RA;
$AID_d \leftarrow$ generate unique anonymous identity for drone;
RA sends AID_d to $\mathcal{D}$;
RA saves $(AID_d, ID_d, (\mathcal{C}_d, \mathcal{R}_d), (\mathcal{C}_d^{sync}, \mathcal{R}_d^{sync})$;
$\mathcal{D}$ saves AID_d in its database;
RSCN Registration:
$\mathcal{C}_r \leftarrow$ generate random challenge;
$\mathcal{C}_r^{sync} \leftarrow$ generate set of re-synchronization challenges;
RA sends $\mathcal{C}_r$ and $\mathcal{C}_r^{sync}$ to RSCN;
$\mathcal{R}_r \leftarrow \mathcal{P}_r(\mathcal{C}_r)$;
$\mathcal{R}_r^{sync} \leftarrow \mathcal{P}_r(\mathcal{C}_r^{sync})$;
RSCN sends $\{\mathcal{R}_r, \mathcal{R}_r^{sync}\}$ to RA;
$UID_r \leftarrow$ generate unique identity for RSCN;
RA sends UID_r to RSCN;
RA saves $UID_r, (\mathcal{C}_r, \mathcal{R}_r), (\mathcal{C}_r^{sync}, \mathcal{R}_r^{sync})$;
RSCN saves UID_r in its database;
Smart Vehicle Registration:
$\mathcal{C}_{sv} \leftarrow$ generate random challenge;
$\mathcal{C}_{sv}^{sync} \leftarrow$ generate set of re-synchronization challenges;
RA sends $\mathcal{C}_{sv}$ and $\mathcal{C}_{sv}^{sync}$ to *SV*;
$\mathcal{R}_{sv} \leftarrow \mathcal{P}_{sv}(\mathcal{C}_{sv})$;
$\mathcal{R}_{sv}^{sync} \leftarrow \mathcal{P}_{sv}(\mathcal{C}_{sv}^{sync})$;
SV sends $\{\mathcal{R}_{sv}, \mathcal{R}_{sv}^{sync}\}$ to RA;
$AID_{sv} \leftarrow$ generate unique anonymous identity for SV;
RA sends AID_{sv} to *SV*;
RA saves $(AID_{sv}, ID_{sv}, (\mathcal{C}_{sv}, \mathcal{R}_{sv}), (\mathcal{C}_{sv}^{sync}, \mathcal{R}_{sv}^{sync})$;
SV saves AID_{sv} in its database;
$\mathcal{B}_{sv} \leftarrow$ bio-metric thumb impression inserted by driver;
$pwd_{sv} \leftarrow$ password chosen by the driver;
$\mathbf{B}_{sv} \leftarrow \mathcal{P}_{sv}(\mathcal{B}_{sv})$;
$PWD_{sv}^{comb} = h(\mathbf{B}_{sv}||pwd_{sv})$;

sends AID_{sv} to the corresponding *SV* and saves $\{(AID_{sv}, ID_{sv}, (\mathcal{C}_{sv}, \mathcal{R}_{sv})$, and $(\mathcal{C}_{sv}^{sync}, \mathcal{R}_{sv}^{sync})\}$.

- Simultaneously, *SV* saves $\{AID_{sv}\}$ in its database. Further, the driver of *SV* inserts the bio-metric thumb impression $\mathcal{B}_{sv}$ into PUF device. Now, a unique unclonable bio-metric impression is generated by the PUF function as: $\mathbf{B}_{sv} = \mathcal{P}_{sv}(\mathcal{B}_{sv})$. This extracted bio-metric impression is now combined with the password pwd_{sv} that is selected by the driver. Thus, the combined strong password is generated as:

$$PWD_{comb} = h(\mathbf{B}_{sv}||pwd_{sv})$$

B. Authentication Process

The authentication process between SV, drones, and RSCN is described in this section. Before explaining the authentication process, the Fig. 3 illustrates the request handling mechanism. In this scenario, both drones and RSCNs have limited storage capacity, meaning they can only process a finite number of requests at a time. Initially, the RSCN accepts and processes requests from SVs sequentially, as indicated by the numbers "1" through "n". However, when the number of requests exceeds the buffer capacity of the RSCN (i.e., at the $(n+1)$th request), the system dynamically redirects the excess requests to the drone within the same zone. The drone then handles these additional requests, starting from the $(n+1)$th up to the $(n+m)$th request. Once the drone's buffer also reaches its capacity, subsequent requests (starting from the $(n+m+1)$th) are redirected back to the RSCN. By this time, the RSCN would have processed its initial requests and freed up buffer space, allowing it to continue processing new incoming requests. This cyclical buffer management process ensures that the system can handle an exceptional number of authentication requests without disruption, maintaining smooth operations even when the number of vehicles on the road exceeds the typical capacity limits.

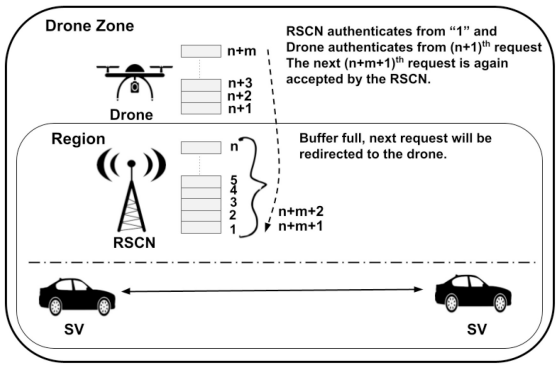


Fig. 3. Requests Handling Process of RSCN and Drone during Authentication.

The following is the mutual authentication process between drone-RSCN and RSCN-SV:

1) *Mutual Authentication Between Drone and RSCN:*

Step 1: The drone generates a random nonce η_d and sends a message M_1 to the RA (Registration Authority) containing η_d and its Anonymous Identifier AID_d.

Step 2: Upon receiving M_1, the RA generates its own random nonce η_{ra}. The RA computes $\eta'_{ra} = \eta_{ra} \oplus \mathcal{R}_d$ and $\delta_1 = h(\eta'_{ra}\|\mathcal{R}_d\|\eta_d)$. The RA sends M_2 to the drone, which includes the encrypted values $\mathcal{C}_d$, η'_{ra}, and δ_1.

Step 3: The drone computes $\mathcal{R}_d = \mathcal{P}_d(\mathcal{C}_d)$ and $\delta_1^* = h(\eta'_{ra}\|\mathcal{R}_d\|\eta_d)$. If δ_1^* matches δ_1, the drone proceeds; otherwise, the process terminates.

Step 4: The drone computes $\eta^*_{ra} = \eta'_{ra} \oplus \mathcal{R}_d$. It also computes $\mathcal{C}_d^{new} = h(\mathcal{C}_d\|\mathcal{R}_d)$ and $\mathcal{R}_d^{new} = \mathcal{P}_d(\mathcal{C}_d^{new})$. The drone then computes $\mathcal{R}'_d = h(ID_d\|\mathcal{R}_d) \oplus \mathcal{R}_d^{new}$ and $UID'_r = h(ID_d\|\eta_{ra}) \oplus UID_r$. Finally, the drone computes $\delta_2 = h(\mathcal{R}'_d\|\mathcal{R}_d\|\eta_{ra}\|UID'_r)$ and sends M_3 to the RA containing $\mathcal{R}'_d$, UID'_r, and δ_2.

Step 5: The RA computes $\delta_2^* = h(\mathcal{R}'_d\|\mathcal{R}_d\|\eta_{ra}\|UID'_r)$. If δ_2^* matches δ_2, the RA proceeds.

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

Step 6: The RA selects a random number a_1 and computes $a_1' = a_1 \oplus \mathcal{R}_r$. It also computes the session key $\mathcal{SK}_r = h(\mathcal{R}_r \| a_1) \oplus \mathcal{SK}$. The RA sends M_4 to the RSCN, which contains AID_d, a_1, $\mathcal{SK}_r$, $\mathcal{C}_r$, and δ_3.

Step 7: The RSCN computes $\mathcal{R}_r = \mathcal{P}_r(\mathcal{C}_r)$ and $\delta_3^* = h(a_1' \| \mathcal{R}_r \| \mathcal{SK}_r \| AID_d)$. If δ_3^* matches δ_3, the RSCN proceeds. The RSCN sends M_5 to the RA, which includes $\mathcal{R}_r'$ and δ_4.

Step 8: The RA computes $\delta_4^* = h(\mathcal{R}_r' \| \mathcal{R}_r \| a_1)$. If δ_4^* matches δ_4, the RA proceeds. The RA generates AID_d^{new} and computes $\mathcal{SK}_d = h(ID_d \| \mathcal{R}_d \| \eta_{ra}) \oplus \mathcal{SK}$. It also computes $AID_d' = h(ID_d \| \mathcal{R}_d \| AID_d) \oplus AID_d^{new}$ and $\delta_5 = h(AID_d' \| \mathcal{SK}_d \| \mathcal{R}_d)$. The RA sends M_6 to the drone containing AID_d', $\mathcal{SK}_d$, and δ_5.

Step 9: The drone computes $\delta_5^* = h(AID_d' \| \mathcal{SK}_d \| \mathcal{R}_d)$. If δ_5^* matches δ_5, the drone computes the session key $\mathcal{SK} = h(ID_d \| \mathcal{R}_d \| \eta_{ra}) \oplus \mathcal{SK}_d$ and generates a new AID_d^{new}. Finally, the drone saves AID_d^{new} in its database, completing the mutual authentication process.

2) *Mutual Authentication Between RSCN and SV:*

Step 1: The SV generates a random nonce η_{sv} and sends a message M_7 to the RA (Registration Authority) containing η_{sv} and its Anonymous Identifier AID_{sv}.

Step 2: Upon receiving M_7, the RA generates its own random nonce η_{ras}. It computes $\eta_{ras}' = \eta_{ras} \oplus \mathcal{R}_{sv}$ and $\delta_6 = h(\eta_{ras}' \| \mathcal{R}_{sv} \| \eta_{sv})$. RA sends M_8 to the SV, which includes the encrypted values $\mathcal{C}_{sv}$, η_{ras}', and δ_6.

Step 3: The SV computes $\mathcal{R}_{sv} = \mathcal{P}_{sv}(\mathcal{C}_{sv})$ and $\delta_6^* = h(\eta_{ras}' \| \mathcal{R}_{sv} \| \eta_{sv})$. If δ_6^* matches δ_6, the SV proceeds; otherwise, the process terminates.

Step 4: The SV computes $\eta_{ras}^* = \eta_{ras}' \oplus \mathcal{R}_{sv}$. It also computes $\mathcal{C}_{sv}^{new} = h(\mathcal{C}_{sv} \| \mathcal{R}_{sv})$ and $\mathcal{R}_{sv}^{new} = \mathcal{P}_{sv}(\mathcal{C}_{sv}^{new})$. It then computes $\mathcal{R}_{sv}' = h(ID_{sv} \| \mathcal{R}_{sv}) \oplus \mathcal{R}_{sv}^{new}$ and $UID_r' = h(ID_{sv} \| \eta_{ras}) \oplus UID_r$. Finally, the SV computes $\delta_7 = h(\mathcal{R}_{sv}' \| \mathcal{R}_{sv} \| \eta_{ras} \| UID_r')$ and sends M_9 to the RA containing $\mathcal{R}_{sv}'$, UID_r', and δ_7.

Step 5: The RA computes $\delta_7^* = h(\mathcal{R}_{sv}' \| \mathcal{R}_{sv} \| \eta_{ras} \| UID_r')$. If δ_7^* matches δ_7, the RA proceeds.

Step 6: The RA selects a random number $a1$ and computes $a1' = a1 \oplus \mathcal{R}_r$. It also computes the session key $\mathcal{SK}_r = h(\mathcal{R}_r \| a1) \oplus \mathcal{SK}$. The RA sends M_{10} to the RSCN, which contains AID_{sv}, $a1$, $\mathcal{SK}_r$, $\mathcal{C}_r$, and δ_8.

Step 7: The RSCN computes $\mathcal{R}_r = \mathcal{P}_r(\mathcal{C}_r)$ and $\delta_8^* = h(a1' \| \mathcal{R}_r \| \mathcal{SK}_r \| AID_{sv})$. If δ_8^* matches δ_8, the RSCN proceeds. The RSCN sends M_{11} to the RA, which includes $\mathcal{R}_r'$ and δ_9.

Step 8: The RA computes $\delta_9^* = h(\mathcal{R}_r' \| \mathcal{R}_r \| a1)$. If δ_9^* matches δ_9, the RA proceeds. The RA generates AID_{sv}^{new} and computes $\mathcal{SK}_{sv} = h(ID_{sv} \| \mathcal{R}_{sv} \| \eta_{ras}) \oplus \mathcal{SK}$. It also computes $AID_{sv}' = h(ID_{sv} \| \mathcal{R}_{sv} \| AID_{sv}) \oplus AID_{sv}^{new}$ and $\delta_{10} = h(AID_{sv}' \| \mathcal{SK}_{sv} \| \mathcal{R}_{sv})$. The RA sends M_{12} to the SV containing AID_{sv}', $\mathcal{SK}_{sv}$, and δ_{10}.

Step 9: The SV computes $\delta_{10}^* = h(AID_{sv}' \| \mathcal{SK}_{sv} \| \mathcal{R}_{sv})$. If δ_{10}^* matches δ_{10}, the SV computes the session key $\mathcal{SK} = h(ID_{sv} \| \mathcal{R}_{sv} \| \eta_{ras}) \oplus \mathcal{SK}_{sv}$ and generates a new AID_{sv}^{new}. Finally, the SV saves AID_{sv}^{new} in its database, completing the mutual authentication process.

VI. Security Analysis

This section outlines both the informal and formal security analysis of the proposed *SECURE* protocol. The informal security analysis evaluates the protocol's resilience against a range of security threats. In contrast, the formal security analysis is conducted using the ROM model, which ensures the robustness of the session key against both active and passive adversaries.

A. Informal Security Analysis

The informal security analysis methodically addresses the core security aspects of the SECURE protocol, including authentication, confidentiality, integrity, non-repudiation, privacy preservation, resistance to various attacks, and the assurance of forward and backward secrecy. By employing formal cryptographic definitions and leveraging well-established mathematical models, we provide a comprehensive justification for the security guarantees offered by the protocol.

1) *Authentication and Confidentiality:* The *SECURE* protocol employs PUF for generating unique cryptographic keys. The uniqueness of the PUF output R_i for a given challenge C_i is formally expressed as:

$$R_i = \mathrm{PUF}(C_i)$$

where R_i is the response generated by the PUF. Given the physical characteristics of each device, no two devices should produce the same response for the same challenge:

$$\Pr(\mathrm{PUF}_A(C_i) = \mathrm{PUF}_B(C_i)) \approx 0$$

for any two devices *A* and *B*. This property ensures that only the legitimate device can generate the correct response to the challenge issued by the network, thereby guaranteeing authentication and confidentiality.

2) *Integrity and Non-repudiation:* Data integrity in *SECURE* is maintained using cryptographic hash functions. Let *h*(*m*) denote the hash of message *m*. The integrity of the message is preserved if:

$$h(m_1) \neq h(m_2) \quad \text{for any} \quad m_1 \neq m_2$$

This property, known as collision resistance, ensures that any alteration of the message during transmission will be detected because the hash of the altered message will not match the original hash.

Non-repudiation is achieved by binding the message *m* to the sender's identity *ID* using a digital signature:

$$\sigma = \mathrm{Sign}_{sk}(h(m) \,||\, ID),$$

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

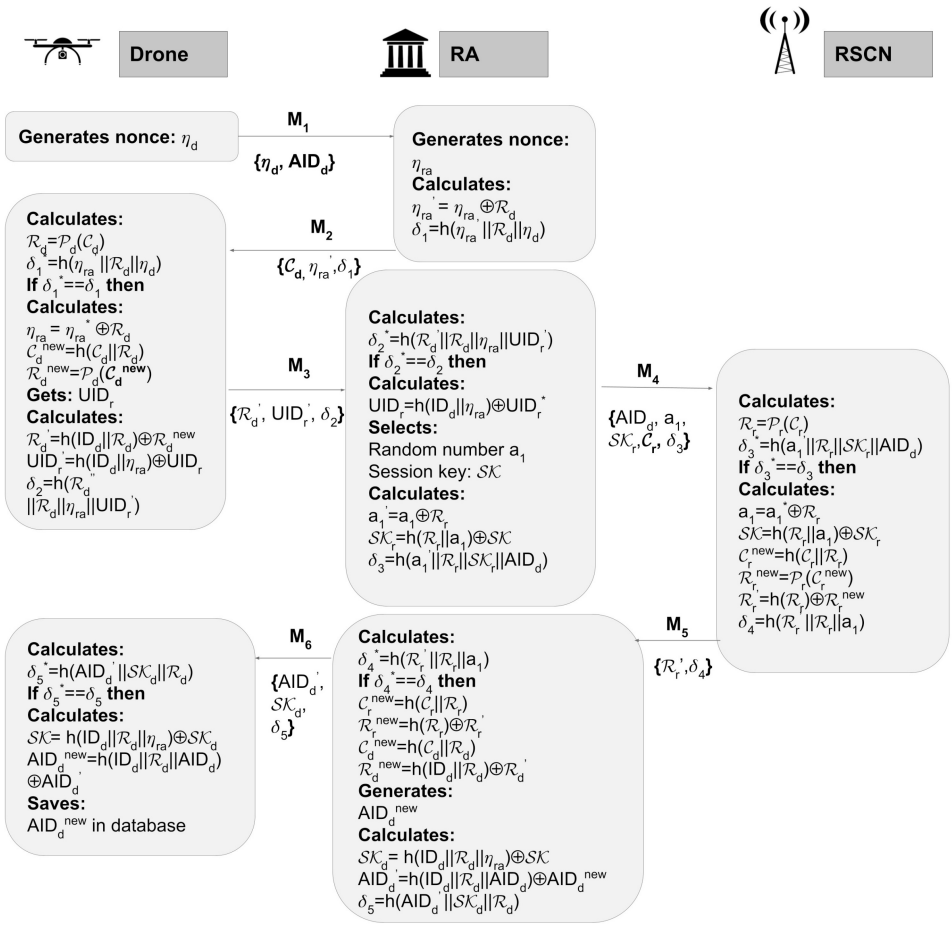


Fig. 4. Mutual Authentication between Drone and RSCN.

where Sign_{sk} is the signing operation with the sender's private key *sk*. The receiver can verify the signature using the sender's public key *pk*:

$$\mathrm{Vrfy}_{pk}(\sigma, h(m) \,||\, ID) = \mathrm{True}$$

This guarantees that the sender cannot deny having sent the message *m* because only the sender's private key could have generated σ.

3) *Privacy Preservation:* In the SECURE protocol, privacy is preserved through the use of pseudonyms and anonymous identities *AID*. Each device has a unique anonymous identity generated during the registration phase:

$$AID_i = h(ID_i \,||\, R_i \,||\, \eta)$$

where ID_i is the real identity of the device, R_i is the PUF response, and η is a nonce. Even if an adversary intercepts AID_i, it is computationally infeasible to reverse-engineer the real identity ID_i without knowing the secret parameters R_i and η. The security of the anonymous identity can be represented as:

$$\Pr(ID_i \,|\, AID_i) \leq \frac{1}{2^b}$$

where b is the bit-length of the cryptographic output, ensuring strong privacy preservation.

4) *Replay Attack Resistance:* To prevent replay attacks, the SECURE protocol incorporates nonces η and timestamps T in its messages. A typical authentication message in SECURE might look like:

$$M = \{ID, \eta, T, h(ID \,||\, \eta \,||\, T)\}$$

To protect against replay attacks, the protocol ensures that the nonce η and timestamp T are unique for each session. The receiver checks the freshness of the timestamp and verifies the nonce before processing the message:

$$\Pr(\text{Replay Attack}) = \Pr\big(\eta' = \eta\big) \times \Pr\big(T' = T\big) \approx 0$$

where η' and T' are the nonce and timestamp of a previously intercepted message. The probability of replaying the exact same message is negligible.

5) *Resistance to Man-in-the-Middle Attacks: SECURE* protects against Man-in-the-Middle (MitM) attacks by enforcing mutual authentication between devices. Let *K* be a shared secret key established during the mutual authentication process:

$$K = h(R_{sv} \,||\, R_{rscn} \,||\, \eta_{sv} \,||\, \eta_{rscn})$$

where R_{sv} and R_{rscn} are the PUF responses of SV and the RSCN, and η_{sv} and η_{rscn} are nonces generated by

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

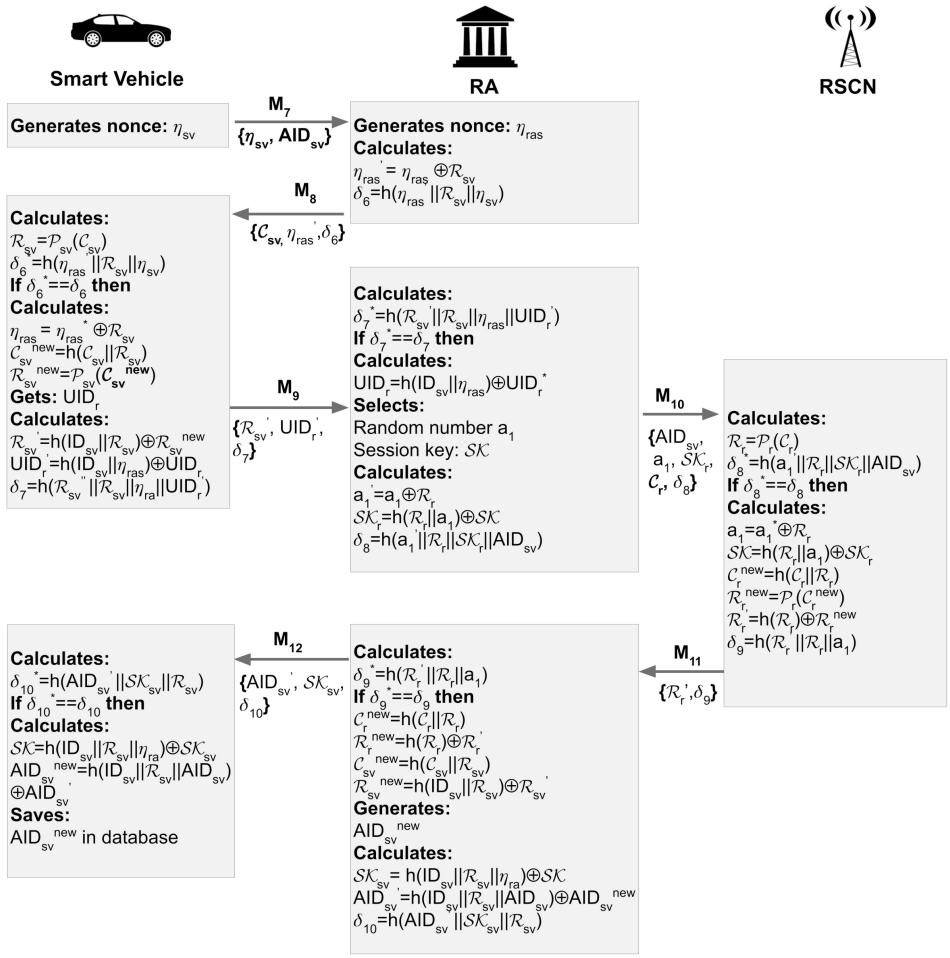


Fig. 5. Mutual Authentication between SV and RSCN.

SV and RSCN. The session key K is used to encrypt subsequent communications:

$$C = E_K(M)$$

Any MitM attacker would need to derive the session key K to decrypt the communication, but this is computationally infeasible without knowledge of the nonces and PUF responses. Thus:

$$\Pr(\text{MitM Attack Success}) \leq \frac{1}{2^{|K|}}$$

where $|K|$ is the bit length of the session key, ensuring robust resistance to MitM attacks.

6) *Forward and Backward Secrecy:* Forward secrecy ensures that the compromise of long-term keys does not compromise past session keys. In SECURE, forward secrecy is maintained by generating new session keys K for each communication session based on nonce and PUF responses:

$$K_{\text{new}} = h(R_{sv}^{\text{new}} \,||\, R_{rscn}^{\text{new}} \,||\, \eta_{sv}^{\text{new}} \,||\, \eta_{rscn}^{\text{new}})$$

Even if an attacker compromises a device and obtains R_{sv}^{old} and R_{rscn}^{old}, they cannot use this information to derive K_{new}.

Backward secrecy is similarly ensured by not storing old session keys once they are used, and by renewing the keys and nonces for each session.

7) *Resistance to Brute-Force Attacks:* The resistance of SECURE to brute-force attacks stems from the use of cryptographic primitives with large key spaces. The probability of successfully guessing a correct key or nonce through brute-force is:

$$\Pr(\text{Brute-Force Success}) = \frac{1}{2^{|K|}} + \frac{1}{2^{|\eta|}}$$

where $|K|$ and $|\eta|$ are the bit lengths of the key and nonce, respectively. By selecting sufficiently large key and nonce sizes, SECURE minimizes the likelihood of a successful brute-force attack.

Table IV provides a detailed security comparison between the proposed *SECURE* protocol and several state-of-the-art protocols, evaluating each across eight critical security features. These features include Authentication (S1), Confidentiality (S2), Integrity & Non-repudiation (S3), Privacy Preservation (S4), Replay Attack Resistance (S5), MitM Resistance (S6), Forward & Backward Secrecy (S7), and Brute-Force Resistance (S8).

The *SECURE* protocol demonstrates comprehensive security, receiving checkmarks across all eight categories,

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
SECURITY COMPARISON OF SECURE WITH OTHER PROTOCOLS

Scheme	S1	S2	S3	S4	S5	S6	S7	S8
[21]	✓	✓	✓	×	✓	✓	✓	✓
[22]	✓	×	×	×	×	×	×	×
[23]	✓	✓	×	✓	×	×	×	×
[24]	✓	✓	×	×	×	×	×	×
[26]	✓	✓	✓	✓	×	✓	×	✓
[27]	✓	✓	×	✓	×	×	×	✓
[28]	✓	✓	✓	✓	×	✓	×	✓
ours	✓	✓	✓	✓	✓	✓	✓	✓

indicating its robustness and suitability for deployment in Drone-assisted Internet of Vehicles (DIoV) environments. *SECURE*'s strength lies in its integration of Physically Unclonable Functions (PUFs) and edge computing, which not only enhance authentication but also ensure confidentiality, integrity, and privacy. The protocol's ability to resist replay and MitM attacks, coupled with its support for forward and backward secrecy, further solidifies its position as a highly secure solution. Additionally, *SECURE*'s resilience against brute-force attacks, achieved through the use of large key spaces and PUF-based defenses, is particularly noteworthy. In contrast, other protocols show varying degrees of security across these features. For instance, while [21] and [26] also demonstrate strong security profiles, protocols like [22] and [23] reveal weaknesses in areas such as integrity & non-repudiation, privacy preservation, and replay attack resistance. These vulnerabilities highlight potential risks that *SECURE* successfully mitigates.

B. Formal Security Analysis

In *SECURE*, the formal security analysis is done using ROM [30]. The ROM model is a formal theoretical model that is utilized to analyze and prove the validity of a cryptographic protocol. The ROM model was chosen for the formal security analysis of *SECURE* due to several key reasons. It is widely recognized in the cryptographic community, which lends credibility and trust to our analysis. This model rigorously tests whether an adversary can distinguish between a real session key and a random value, essential for ensuring strong confidentiality and integrity. The ROM model also enables direct comparisons with other protocols analyzed using the same model, highlighting the improvements and advantages of *SECURE*. Additionally, it is flexible and comprehensive, capable of simulating various attack scenarios, including advanced ones like man-in-the-middle attacks. The ROM model supports formal security proofs, showing that any adversary's advantage is negligible, and its effective use in many related works further validates our choice. By using the ROM model, our security analysis is robust, credible, and aligned with leading research in the field. In this section, we have proved that it is challenging for $\mathcal{A}$ to retrieve the session key used in *SECURE*. Here, we denote the instances of SV, RSCN, and drones as $\mathcal{I}_{sv}^{a_1}$, $\mathcal{I}_r^{a_2}$, and $\mathcal{I}_d^{a_3}$. We assume that $\mathcal{A}$ can access and monitor all the public channel communication between these entities.

In *SECURE*, $\mathcal{A}$ can perform the following queries or tasks:

- $Corrupt(\mathcal{I}_{sv}^{a_1})$*:* $\mathcal{A}$ can illegally access the secret credentials such as user identity, passwords of the participant $\mathcal{I}_{sv}^{a_1}$.
- $Send(\mathcal{I}_x^{a_n}, M)$*:* Message, *M* is sent to $\mathcal{I}_x^{a_n}$ and if *M* is valid, $\mathcal{A}$ is considered to be a legitimate user.
- $Perform(\mathcal{I}_{sv}^{a_1}, \mathcal{I}_r^{a_2}, \mathcal{I}_d^{a_3})$*:* $\mathcal{A}$ can access the message transmitted via public channel between the three entities and performs passive and active attacks.
- $Expose(\mathcal{I}_x^{a_n})$*:* $\mathcal{A}$ exposes the session key established during the communication between the three entities.
- $Flip(\mathcal{I}_x^{a_n})$*:* $\mathcal{A}$ tosses an unbiased coin and gets the result c_{toss}. Depending on the results, $\mathcal{A}$ determines the following outcomes: $c_{toss} = 1$, the obtained session key is correct, $c_{toss} = 0$, $\mathcal{A}$ obtains random nonce, and if both results are not obtained, $\mathcal{A}$ gets *NULL*.

Theorem 1: The advantage of $\mathcal{A}$, to break the security of *SECURE* for computing the session key *SK* in polynomial time T_p is formulated as:

$$\Gamma_{\mathcal{A}}^{protocol}(T_p) \leq \left(\frac{Query_h^2}{|h()|} + \frac{Query_p^2}{|\mathcal{P}()|} + 2max(Z'.Query_S^{S'}, \frac{Query_S}{2^b}) \right). \tag{1}$$

where, $Query_h, Query_p$, and $Query_S$ are the number of queries for a hash function, PUF function, and sending/receiving a message respectively. Here, b denotes the number of bits in biometric impression $\mathcal{B}$. The Z' and S' are the Zipf's law [31] parameters used in the distribution done using Zipf's law. It is different from the uniform distribution. The $|h()|, |\mathcal{P}()|$ represents the range space of hash functions and PUF functions.

Proof: To prove the security of session key, the following cases are considered:

- *Case 1:* $\mathcal{A}$ performs an attack on *SECURE* for guessing w bits of the password in polynomial time (T_p). Here, w bits are selected in starting of the first attempt; hence, the advantage for $\mathcal{A}$ can be formulated as:
$$\Gamma_{\mathcal{A}}^{protocol}(T_p) = |2.P[Win_1] - 1|. \tag{2}$$
Here, Win_1 denotes the success of $\mathcal{A}$ in guessing the w bits.
- *Case 2:* $\mathcal{A}$ performs an eavesdrop attack by accessing the transmitted messages. Consider the messages as, M_1, M_2, M_3, M_4, M_5, and M_6. It access the session key which is computed as $\mathcal{SK} = h(ID_{sv}||R_{sv}||\eta_{RA}) \oplus \mathcal{SK}_{sv}$, $\mathcal{SK} = h(ID_d||R_d||\eta_{RA}) \oplus \mathcal{SK}_d$, and $\mathcal{SK} = h(R_r||a_1) \oplus \mathcal{SK}_r$ for SV, drones, and RSCN respectively. Here, $\mathcal{A}$ needs to find out the secret parameters like response of *SV* (R_{sv}), response of drone (R_d), response of RSCN (R_r), identity of *SV* (ID_{sv}), and identity of drone (ID_d). These responses and identities are only known to the authenticated users and thus the probability of winning is the same as the case 1 and it can be shown as:
$$P[Win_2] = P[Win_1]. \tag{3}$$
- *Case 3:* $\mathcal{A}$ performs an impersonation attack by pretending itself as an authorized receiver. Since, all the transmitted messages have the secret parameters and the

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

hash function is collision-resistant, the difference between the probability of Win_3 and Win_2 can be formulated as:

$$|P[Win_3] - P[Win_2]| \leq \frac{Query_h^2}{|h()|}. \quad (4)$$

- *Case 4:* Similar to case 3, we can formulate the above equation for secure PUF function as:

$$|P[Win_4] - P[Win_3]| \leq \frac{Query_p^2}{|\mathcal{P}()|}. \quad (5)$$

- *Case 5:* We assume that the OBU and edges are the corrupt devices and $\mathcal{A}$ can access the secret parameters stored in these devices. Due to the property of PUF device, the probability of guessing password is reduced to $\frac{1}{2_b}$, where b is the number of bits in biometric password, $\mathcal{B}_{sv}$. Here, $\mathcal{A}$ can also try to guess the password having less entropy using Zipf's law and we can use those parameters as well. We can formulate the relation as:

$$|P[Win_5] - P[Win_4]| \leq \max\left(Z'.Query_S^{S'}, \frac{Query_S}{2^b}\right). \quad (6)$$

At last, we assume that $\mathcal{A}$ can also perform random guessing of w bits of user password and the probability of winning can be written as:

$$P[Win_5] = \frac{1}{2}. \quad (7)$$

Using the above equations (2), (3), and (6), we get:

$$\frac{1}{2}\Gamma_{\mathcal{A}}^{protocol}(T_p) = |P[Win_2] - P[Win_5]|. \quad (8)$$

Using the above equations (4), (5), and (6), we can get the result as:

$$|P[Win_2 - Win_5]| \leq \frac{Query_h^2}{|h()|} + \frac{Query_p^2}{|\mathcal{P}()|} + 2max\left(Z'.Query_S^{S'}, \frac{Query_S}{2^b}\right). \quad (9)$$

Finally, we get the desired output of (1) using the equation (8) and (9):

$$\Gamma_{\mathcal{A}}^{protocol}(T_p) \leq \frac{Query_h^2}{|h()|} + \frac{Query_p^2}{|\mathcal{P}()|} + 2max\left(Z'.Query_S^{S'}, \frac{Query_S}{2^b}\right). \quad (10)$$

VII. Performance Analysis and Simulation Results

We have used MIRACL and Java 8 to execute different cryptographic operations on Jetson Xavier NX and Raspberry Pi 4 systems. In *SECURE*, we have considered two types of devices for the experiments: NVIDIA Jetson Xavier NX and Raspberry Pi 4 (4 GB & 8 GB). As mentioned in the network model, there are three different entities that are participating in the communication: drone, SV, and RSCN. In our experimental setup for the SECURE protocol, we utilized Raspberry Pi 4 systems for both drones and SV, with the drones equipped with 8 GB LPDDR4-3200 SDRAM and the SVs with 4 GB LPDDR4-3200 SDRAM. The Raspberry Pi 4 features a quad-core ARM Cortex-A72 CPU running at 1.5 GHz, supports up to 2 TB of storage via a MicroSD card, and includes connectivity options like Gigabit Ethernet, 802.11ac wireless, and Bluetooth 5.0.

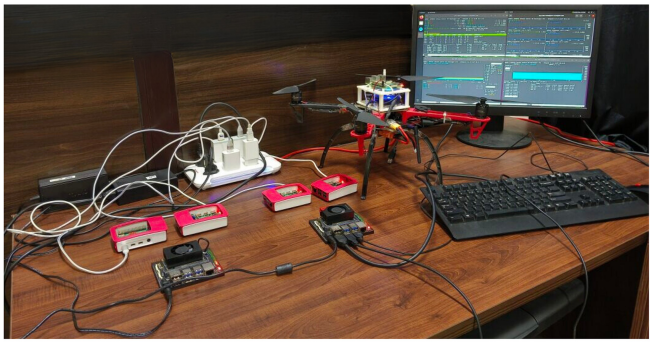


Fig. 6. Experimental Setup for the Execution of *SECURE*.

TABLE V
Execution Time (in ms) of Each Cryptographic Operation on Different Platforms

Cryptographic Operations	Jetson Xavier NX (RSCNs)	Raspberry Pi 4-4GB (SV)	Raspberry Pi 4-8GB (Drone)
T_h	0.005	1.82	1.08
T_{pa}	5.44	12.54	10.23
T_m	3.27	8.44	7.45
T_a	0.43	7.67	7.06
T_e	2.67	10.56	9.46
T_{sm}	1.37	4.86	4.23
T_{puf}	0.56	1.04	0.96

For the RSCNs, we employed the Jetson Xavier NX, which delivers up to 21 Tera Operations Per Second (TOPS) of computational performance while operating under 15W of power. This device is powered by a 6-core NVIDIA Carmel 64-bit ARMv8.2 CPU, an integrated 384-core NVIDIA Volta GPU with 48 Tensor Cores, and 8GB of 128-bit LPDDR4x RAM, providing a bandwidth of 51.2GB/s. The Jetson Xavier NX supports dual 4K displays, Gigabit Ethernet, USB 3.1, PCIe Gen 3, and various I/O interfaces. In addition to these systems, we integrated a 128-bit strong SRAM PUF circuit, which is embedded on an MSP430 microcontroller with a 798 MHz CPU, to enhance the security of the network. The setup described in the Fig. 6 forms the backbone of our experimental environment for executing the *SECURE* protocol.

A. Performance Analysis

For the performance analysis, we have executed the proposed *SECURE* algorithm on different platforms. Table V presents the execution time (in milliseconds) of various cryptographic operations performed on three different hardware platforms: Xavier NX (RSCNs), Raspberry Pi 4 with 4GB RAM (SV), and Raspberry Pi 4 with 8GB RAM (Drone). The cryptographic operations evaluated include hash operations (T_h), point addition on elliptic curves (T_{pa}), modular multiplication (T_m), modular addition (T_a), modular exponentiation (T_e), scalar multiplication (T_{sm}), and Physical Unclonable Function (T_{puf}) operations.

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
CRYPTOGRAPHIC EQUATIONS OF SV, DRONE, AND RSCN

Authentication	Smart Vehicle	Drone	RSCN
Between Drone and RSCN	—	$2T_{puf} + 8T_h$	$2T_{puf} + 4T_h$
Between SV and RSCN	$2T_{puf} + 8T_h$	—	$2T_{puf} + 4T_h$

The Jetson Xavier NX exhibits the fastest execution times across all the evaluated operations, indicating its superior processing capabilities in comparison to the Raspberry Pi models. The Raspberry Pi 4 with 4GB RAM shows moderate performance, with execution times generally longer than those of the Jetson Xavier NX, reflecting its relatively lower processing power. Meanwhile, the Raspberry Pi 4 with 8GB RAM, while similar in performance to the 4GB variant, demonstrates slightly better execution times for some operations, likely due to the additional memory available, which can enhance performance in certain cryptographic tasks.

The table VI summarizes the cryptographic equations used for authentication between different components in the proposed SECURE protocol, specifically focusing on SVs, Drones, and RSCNs. The equations represent the computational effort required for authentication between these entities, expressed in terms of the time taken for specific cryptographic operations. For the authentication process between a Drone and an RSCN, the Drone requires two PUF operations and eight hash operations, represented as $2T_{puf} + 8T_h$. On the other side, the RSCN requires two PUF operations and four hash operations, denoted as $2T_{puf} + 4T_h$. Similarly, for authentication between a SV and an RSCN, the SV performs two PUF operations and eight hash operations, following the same equation as the Drone, $2T_{puf} + 8T_h$. The RSCN, consistent with its role, performs two PUF operations and four hash operations, represented as $2T_{puf} + 4T_h$.

Table VII provides a detailed comparison of the proposed SECURE protocol with other competitive schemes from the literature, including those by Khan et al. (2021) [21], Hussain et al. (2022) [22], Wu et al. (2022) [23], Chaudhry et al. (2021) [24], Wang et al. (2024) [26], Karmakar et al. (2024) [27], and Tian et al. (2024) [28]. The comparison is based on several key parameters: cryptographic equations, computation cost (in milliseconds), communication cost (in bits), energy consumption (in joules), edge computing capability, and usage of PUF.

1) *Computation Cost:* The SECURE protocol achieves a remarkably low computation cost of 4.6 milliseconds, which represents a significant improvement over other schemes. For instance, Wang et al. (2024) [26], which also utilizes PUFs, reports a computation cost of 20.89 milliseconds, making *SECURE* approximately 77.98% faster. Similarly, the protocol by Chaudhry et al. (2021) [24], with a computation cost of 18.76 milliseconds, is around 75.48% slower than *SECURE*. The *SECURE* protocol's computation cost advantage stems from its efficient cryptographic equation, $8T_{puf} + 24T_h$, which leverages the strengths of PUF technology while minimizing the computational overhead. In contrast, Khan et al. (2021) [21] relies solely on modular multiplication $(4T_m)$ but still incurs a higher computation cost of 13.08 milliseconds, further emphasizing the efficiency of *SECURE*.
2) *Communication Cost:* The communication cost is another critical metric where the *SECURE* protocol excels, requiring only 384 bits. This is significantly lower than Wu et al. (2022) [23], which has a communication cost of 6720 bits–approximately 17.5 times higher than *SECURE*. Such a reduction in communication overhead is vital in resource-constrained environments, where bandwidth and energy efficiency are paramount. Even when compared to other efficient protocols like Karmakar et al. (2024) [27], which has a communication cost of 1024 bits, *SECURE* demonstrates a more optimized approach, reducing communication overhead by more than 60%. This efficiency is crucial for real-time applications, particularly in edge computing scenarios where minimizing data transmission is essential.
3) *Energy Consumption:* Energy efficiency is another area where the *SECURE* protocol demonstrates clear superiority. With an energy consumption of 23.0 joules, *SECURE* is significantly more energy-efficient than protocols such as Wang et al. (2024) [26], which consumes 104.45 joules–over 4.5 times more than *SECURE*. The reduced energy consumption directly correlates with the lower computation cost and optimized cryptographic operations in *SECURE*. When compared to Hussain et al. (2022) [22], which consumes 33.075 joules, *SECURE* achieves approximately 30% lower energy consumption. This reduction is critical for battery-powered devices like drones and IoT sensors, where energy efficiency directly impacts operational longevity.
4) *Edge-Computing and PUF Usage:* One of the key differentiators of the *SECURE* protocol is its integration of edge computing and PUF technology. While several protocols such as Wu et al. (2022) [23] and Chaudhry et al. (2021) [24] do not incorporate these technologies, *SECURE* leverages them to enhance both security and performance. The use of PUFs provides device-specific security, mitigating risks such as cloning and tampering, while edge computing reduces latency by processing data closer to its source.

Fig. 7 graphically illustrates the comparison of computation overhead, energy consumption, and communication cost across the different schemes. The *SECURE* protocol (denoted as "ours") is positioned favorably with the lowest computation overhead and energy consumption. This is evident in the stacked bar chart, where *SECURE* consistently outperforms the other schemes. The proximity of the bars for computation overhead and energy consumption in *SECURE* indicates a balanced approach to both performance and energy efficiency. The line plot for communication cost clearly shows that SECURE has one of the lowest communication overheads among the schemes, making it highly suitable for applications where bandwidth is limited or where efficient data transmission is critical.

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
COMPARISON OF *SECURE* WITH OTHER COMPETITIVE SCHEMES

Schemes	Cryptographic Equations	Computation Cost (ms)	Communication Cost (bits)	Energy Consumption (J)	Edge-Computing	PUF Usage
Khan et al. (2021) [21]	$4T_m$	13.08	336	65.4	No	No
Hussain et al. (2022) [22]	$2T_{puf} + 2T_e + 31T_h$	6.615	2061	33.075	No	No
Wu et al. (2022) [23]	$T_m + 2T_e + 30T_h$	8.76	6720	43.8	No	No
Chaudhry et al. (2021) [24]	$13T_h + 7T_e$	18.76	2624	93.8	No	No
Wang et al. (2024) [26]	$2T_{puf} + 31T_h + 6T_m$	20.89	2144	104.45	No	Yes
Karmakar et al. (2024) [27]	$2T_{puf} + 8T_h$	1.16	1024	5.8	No	Yes
Tian et al. (2024) [28]	$2T_{puf} + 16T_h$	1.2	3008	6.0	No	Yes
SECURE (Our Proposal)	$8T_{puf} + 24T_h$	4.6	384	23.0	Yes	Yes

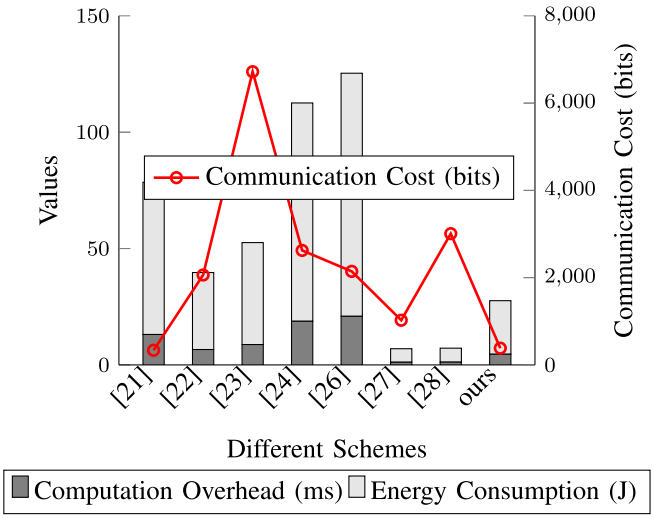


Fig. 7. Comparison of Computation Overhead, Communication Cost, and Energy Consumption for Different Schemes.

TABLE VIII
SIMULATION PARAMETERS AND ITS VALUES

Simulation Parameters	Details
Network simulation tool	NS-2
Mobility tool	SUMO
Channel model	Two-ray interference model
Simulation area	2000 m x 2000 m
Communication range of Drone	1000 m
Communication range of RSCN	500 m
Communication range of SV	300 m
Communication protocol	IEEE 802.11p
Data transmission rate	12 Mb/s
Simulation time	100 seconds
Buffer size for $SECURE$	50 packets
Number of simulations	5

B. Simulation Environment and Results

We have done the simulation of *SECURE* protocol using SUMO and NS2. We have generated the *.osm* file for an urban city in India using *OSMWebWizard* with the help of SUMO simulator for our case study. The generated *.osm* file is converted to the *.cfg* file which is the map configure file. Further, the *.xml* file is generated using a *.cfg* file. In Table VIII, we have shown the simulation parameters and its values. The "*TraceExporter*" function is used to generate a *.tcl* script for vehicle mobility with the help of an existing *.cfg* file. The *SECURE* uses IEEE 802.11p which considers Dedicated Short Range Communication protocol (DSRC) with 5.9 GHz frequency band and 10 MHz control channel.

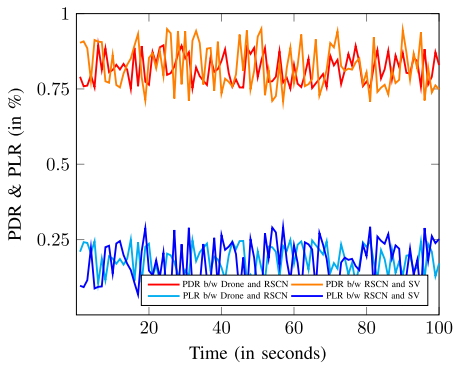


Fig. 8. Observation of PDR and PLR Values for Two Types of Communications: (1) Between Drone and RSCN, (2) Between RSCN and SV.

We explain simulation results in terms of Packet Delivery Ratio (PDR) and Packet Loss Ratio (PLR). A PDR can be measured as the ratio of the number of packets delivered in total to the total number of packets sent from source node to the destination node in a network. In *SECURE*, the PDR value for communication between drone and RSCN varies from 0.75 to 0.90, and between SV and RSCN varies from 0.70 to 0.95. In Fig. 8, we have graphically shown the simulated results. The PLR represents the ratio of the number of lost packets to the total number of sent packets. Each packet has a deadline before which it must be executed, and if this is not possible, the scheduler tries to minimize the number of lost packets due to deadline expiry. The PLR value for the communication between drone and RSCN varies from 0.1 to 0.25 and between SV and RSCN varies from 0.05 to 0.30. In Fig. 8, we have graphically shown the simulated results.

VIII. CONCLUSION AND FUTURE WORKS

In this research paper, we introduce a novel protocol called *SECURE* for authentication and preserving privacy in a DIoVs environment. By leveraging edge computing and PUFs, *SECURE* offers a secure and efficient solution for DIoVs networks, overcoming the limitations of traditional RSU-based systems. The integration of UAVs as mobile edge nodes provides extended coverage, lower latency, and increased adaptability to the DIoVs network. The experimental results demonstrate the effectiveness of *SECURE* in enhancing authentication, privacy, and scalability compared to existing DIoVs networks. Notable improvements are observed in communication cost, computation cost at the SV level, RSCN efficiency, and drone-side operations. The *SECURE* showcases 5 times less communication cost and 2 times in computation cost. These results emphasize the practical benefits of

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.

SECURE in terms of enhanced performance, resilience, and efficiency. As part of future work, we will include storage cost and other network parameters (end-to-end delay and jitter).

REFERENCES

[1] G. H. Shill. "Public comment on request for comments on the national highway traffic safety administration's new car assessment program." May 26, 2022. [Online]. Available: https://ssrn.com/abstract=4120803

[2] M. Hussein et al., "Key technologies for safe and autonomous drones," *Microprocess. Microsyst.*, vol. 87, Nov. 2021, Art. no. 104348.

[3] V. O. Nyangaresi and N. Petrovic, "Efficient PUF based authentication protocol for Internet of Drones," in *Proc. Int. Telecommun. Conf. (ITC-Egypt)*, 2021, pp. 1–4.

[4] O. Kaiwartya et al., "Internet of Vehicles: Motivation, layered architecture, network model, challenges, and future aspects," *IEEE Access*, vol. 4, pp. 5356–5373, 2016.

[5] W. Shi, H. Zhou, J. Li, W. Xu, N. Zhang, and X. Shen, "Drone assisted vehicular networks: Architecture, challenges and opportunities," *IEEE Netw.*, vol. 32, no. 3, pp. 130–137, May/Jun. 2018.

[6] M. Tanveer, A. Aldosary, S.-U.-D. Khokhar, A. K. Das, S. A. Aldossari, and S. A. Chaudhry, "PAF-IoD: PUF-enabled authentication framework for the Internet of Drones," *IEEE Trans. Veh. Technol.*, vol. 73, no. 7, pp. 9560–9574, Jul. 2024.

[7] A. Irshad, B. A. Alzahrani, A. Albeshri, K. Alsubhi, A. Nayyar, and S. A. Chaudhry, "SPAKE-DC: A secure PUF enabled authenticated key exchange for 5G-based drone communications," *IEEE Trans. Veh. Technol.*, vol. 73, no. 4, pp. 5770–5780, Apr. 2024.

[8] J. Miao, Z. Wang, X. Ning, A. Shankar, C. Maple, and J. J. P. C. Rodrigue, "A UAV-assisted authentication protocol for Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 10286–10297, Aug. 2024.

[9] J. Zheng, H. Duan, C. Wang, Q. Cao, G. Xu, and R. Fang, "A drone-assisted anonymous authentication and key agreement protocol with access control for accident rescue in the Internet of Vehicles," *Electronics*, vol. 13, no. 10, p. 1939, 2024.

[10] L. Gupta, R. Jain, and G. Vaszkun, "Survey of important issues in UAV communication networks," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1123–1152, 2nd Quart., 2016.

[11] C. Herder, M.-D. Yu, F. Koushanfar, and S. Devadas, "Physical unclonable functions and applications: A tutorial," *Proc. IEEE*, vol. 102, no. 8, pp. 1126–1141, Aug. 2014.

[12] M. A. El-Zawawy, A. Brighente, and M. Conti, "Authenticating drone-assisted Internet of Vehicles using elliptic curve cryptography and blockchain," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1775–1789, Jun. 2023.

[13] M. Turkanović, B. Brumen, and M. Hölbl. "A novel user authentication and key agreement scheme for heterogeneous Ad Hoc wireless sensor networks, based on the Internet of Things notion," *Ad Hoc Networks*, vol. 20, pp. 96–112, Sep. 2014.

[14] M. S. Farash, M. Turkanovic, S. Kumari, and M. Hölbl, "An efficient user authentication and key agreement scheme for heterogeneous wireless sensor network tailored for the Internet of Things environment," *Ad Hoc Netw.*, vol. 36, pp. 152–176, Jan. 2016.

[15] S. Challa et al., "Secure signature-based authenticated key establishment scheme for future IoT applications," *IEEE Access*, vol. 5, pp. 3028–3043, 2017.

[16] Z. Zhang, Y. Huang, W. Huang, W. Pan, Y. Liao, and S. Zhou, "An auto-upgradable end-to-end pre-authenticated secure communication protocol for UAV-aided perception intelligent system," *IEEE Internet Things J.*, vol. 11, no. 18, pp. 30187–30203, Sep. 2024.

[17] I. Bhattarai, C. Pu, and K. R. Choo, "A lightweight aggregate authentication protocol for Internet of Drones," in *Proc. IEEE 21st Consum. Commun. Netw. Conf. (CCNC)*, 2024, pp. 143–151.

[18] S. M. Gilani, A. Anjum, A. Khan, M. H. Syed, S. A. Moqurrab, and G. Srivastava, "A robust Internet of Drones security surveillance communication network based on IOTA," *Internet Things*, vol. 25, Apr. 2024, Art. no. 101066.

[19] J. Won, S.-H. Seo, and E. Bertino, "Certificateless cryptographic protocols for efficient drone-based smart city applications," *IEEE Access*, vol. 5, pp. 3721–3749, 2017.

[20] M. Wazid, A. K. Das, N. Kumar, A. V. Vasilakos, and J. J. P. C. Rodrigues, "Design and analysis of secure lightweight remote user authentication and key agreement scheme in Internet of Drones deployment," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 3572–3584, Apr. 2019.

[21] M. A. Khan et al., "A provable and privacy-preserving authentication scheme for UAV-enabled intelligent transportation systems," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3416–3425, May 2022.

[22] S. Hussain, K. Mahmood, M. K. Khan, C. M. Chen, B. A. Alzahrani, and S. A. Chaudhry, "Designing secure and lightweight user access to drone for smart city surveillance," *Comput. Stand. Interfaces*, vol. 80, Mar. 2022, Art. no. 103566.

[23] F. Wu, X. Li, X. Luo, and K. Gu, "A novel authentication scheme for edge computing-enabled Internet of Vehicles providing anonymity and identity tracing with drone-assistance," *J. Syst. Archit.*, vol. 132, Nov. 2022, Art. no. 102737.

[24] S. A. Chaudhry et al., "A lightweight authentication scheme for 6G-IoT enabled maritime transport system," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2401–2410, Feb. 2023.

[25] U. Chatterjee et al., "Building PUF based authentication and key exchange protocol for IoT without explicit CRPs in verifier database," *IEEE Trans. Dependable Secure Comput.*, vol. 16, no. 3, pp. 424–437, May/Jun. 2019.

[26] D. Wang, Y. Cao, K.-Y. Lam, Y. Hu, and O. Kaiwartya, "Authentication and key agreement based on three factors and PUF for UAVs-assisted post-disaster emergency communication," *IEEE Internet Things J.*, vol. 11, no. 11, pp. 20457–20472, Jun. 2024.

[27] R. Karmakar, G. Kaddoum, and O. Akhrif, "A PUF and fuzzy extractor-based UAV-ground station and UAV-UAV authentication mechanism with intelligent adaptation of secure sessions," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 3858–3875, May 2024.

[28] C. Tian, J. Ma, T. Li, J. Zhang, C. Ma, and N. Xi, "Provably and physically secure UAV-assisted authentication protocol for IoT devices in unattended settings," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4448–4463, 2024.

[29] D. Basin and C. Cremers, "From Dolev-Yao to strong adaptive corruption: Analyzing security in the presence of compromising adversaries," Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep. 2009/079, 2009.

[30] E. Alkim et al., "Revisiting TESLA in the quantum random oracle model," in *Proc. Int. Workshop Post-Quantum Cryptogr.*, 2017, pp. 143–162.

[31] D. Wang, H. Cheng, P. Wang, X. Huang, and G. Jian, "Zipf's law in passwords," *IEEE Trans. Inf. Forensics Security*, vol. 12, pp. 2776–2791, 2017.

Authorized licensed use limited to: University of Melbourne. Downloaded on November 03,2026 at 12:44:10 UTC from IEEE Xplore. Restrictions apply.